

Lab 2: Sampling, Aliasing and Quantization

Setup

```
!pip install wfdb

import wfdb
import numpy as np
import matplotlib.pyplot as plt

record = wfdb.rdrecord('100', pn_dir='mitdb', sampio=3600)
fs = record.fs
ecg = record.p_signal[:, 0]
t = np.arange(len(ecg)) / fs
```

1. Visualize the signal

```
plt.figure(figsize=(12, 4))
plt.plot(t, ecg)
plt.title('ECG'); plt.xlabel('Time (s)'); plt.ylabel('mV')
plt.grid(True); plt.show()

# Zoom to 1 second to see individual samples
plt.figure(figsize=(12, 4))
plt.stem(t[:int(fs)], ecg[:int(fs)], basefmt=' ')
plt.title('First second'); plt.grid(True); plt.show()
```

Task 1. Identify the QRS spikes in the full plot.

Task 2. Count the samples between two beats in the stem plot.

2. Resample

Lower the rate by keeping every k-th sample. New rate = 360 / k.

```
factor = 4
ecg_ds = ecg[::factor]
fs_ds = fs / factor
t_ds = np.arange(len(ecg_ds)) / fs_ds

plt.figure(figsize=(12, 4))
plt.plot(t, ecg, 'gray', alpha=0.5, label='360 Hz')
plt.plot(t_ds, ecg_ds, 'k', label=f'{fs_ds:.0f} Hz')
plt.legend(); plt.grid(True); plt.show()
```

Task 3. Run with factor = 4.

Task 4. Try factor = 2, 8, 20. Note where the ECG shape breaks down.

3. Undersampling and aliasing

A 50 Hz tone sampled below 100 Hz appears as a false lower frequency.

```
f_tone = 50
tt = np.arange(0, 0.2, 1/2000)
tone = np.sin(2*np.pi*f_tone*tt)
```

```

fs_bad = 60 # < 2 x 50 -> aliasing
nb = np.arange(0, 0.2, 1/fs_bad)
tone_bad = np.sin(2*np.pi*f_tone*nb)

plt.figure(figsize=(12, 4))
plt.plot(tt, tone, 'gray', alpha=0.6, label='True 50 Hz')
plt.plot(nb, tone_bad, 'ko-', label=f'{fs_bad} Hz')
plt.legend(); plt.grid(True); plt.show()

alias = abs(f_tone - round(f_tone/fs_bad)*fs_bad)
print('Apparent frequency:', alias, 'Hz')

```

Task 5. Run it and read the alias frequency.

Task 6. Try `fs_bad = 40, 55, 80`. Record the alias each time.

Task 7. Find the lowest `fs_bad` with no aliasing.

4. Oversampling and decimation

Low-pass filter before downsampling, or high frequencies alias into the signal.

```

ecg_noisy = ecg + 0.3*np.sin(2*np.pi*80*t) # add 80 Hz interference

naive = ecg_noisy[::6] # no filter
ecg_lp = np.convolve(ecg_noisy, np.ones(12)/12, mode='same')
decimated = ecg_lp[::6] # filter then downsample
t_new = np.arange(len(naive)) / (fs/6)

plt.figure(figsize=(12, 5))
plt.subplot(2,1,1); plt.plot(t_new, naive, 'k')
plt.title('No filter (aliased)'); plt.grid(True)
plt.subplot(2,1,2); plt.plot(t_new, decimated, 'k')
plt.title('Filtered then decimated'); plt.grid(True)
plt.tight_layout(); plt.show()

```

Task 8. Compare the two plots.

Task 9. Change 80 Hz to 150 Hz and re-run.

5. Quantization and SQNR

N bits give 2^N levels. SQNR rises about 6 dB per bit.

```

def quantize(sig, bits):
    levels = 2 ** bits
    smin, smax = sig.min(), sig.max()
    step = (smax - smin) / (levels - 1)
    return np.round((sig - smin)/step)*step + smin

bits = 3
ecg_q = quantize(ecg, bits)

plt.figure(figsize=(12, 4))
plt.plot(t, ecg, 'gray', alpha=0.5, label='Original')

```

```
plt.plot(t, ecg_q, 'k', label=f'{bits}-bit')
plt.legend(); plt.grid(True); plt.show()

sqnr = 10*np.log10(np.sum(ecg**2) / np.sum((ecg-ecg_q)**2))
print(f'{bits}-bit -> SQNR = {sqnr:.1f} dB')
```

Task 10. Run with bits = 3.

Task 11. Re-run for bits = 4, 6, 8, 10 and fill the table.

Task 12. Check whether SQNR rises about 6 dB per bit.

Results table:

Bits (N)	Levels (2^N)	SQNR (dB)
3		
4		
6		
8		
10		